A Twin-Gated Control Loop for Campus Wi-Fi: Verifying SDN and LLM-Proposed Actions in a Digital Twin Before They Reach the Network

Aggimalla Abhishek, N. Likhith Naik, Sundaram, Sambhav Mishra, Bikram Hawaldar
Indian Institute of Information Technology Dharwad, India

Jagadeesha R B
Dept. of Electronics and Communication Engineering
Indian Institute of Information Technology Dharwad, India

***Abstract*—A software-defined network (SDN) controller can react to congestion, access-point (AP) failures and interference in campus Wi-Fi, but a change applied directly to the live network shows its effect only afterwards. We present a control loop in which no action, whether proposed by a heuristic optimizer or by a large language model (LLM) that translates an operator's intent, reaches the network without an accepted verdict from a digital twin. Only the heuristic optimizer was run on the emulated network; the LLM is evaluated offline. The twin is an analytical model of per-AP airtime sharing, co-channel interference and M/M/1/K queueing. An executor applies accepted actions, watches the live KPIs and rolls back regressions. On an emulated campus (Mininet-WiFi and Ryu; 4 APs, 20 stations) we compare plain SDN (V1), heuristics without the twin (V2) and the full system (V3, which also holds high-impact actions for an operator) in three scenarios and three seeds each (27 runs). The twin predicts held-out per-flow throughput within 4.5% (4.0% in steady state) and answers in a median 0.61 ms, but in the closed loop its predictions are no closer to the live measurements than "nothing changes" (median throughput error 56.7% against 46.5%). In a lecture flash crowd both acting variants cut the loss of the affected flows from 43.9% to under 10% and more than double their throughput. H1 (V3 recovers 30% faster than V1) is not met on the capped recovery times (V3 faster by 8, 17 and 0%), and in the interference scenario V3 itself never recovered, because its channel change waited for an absent operator. In V2, which applies every action whatever the verdict, the twin rejected only 2 of the 23 actions the watch found harmful, and no stricter acceptance rule fixes that; the watch-and-rollback step gave the main protection and itself failed 3 times. In a separate offline test, an LLM behind schema validation translated 30 of 30 intents correctly; none of its actions was applied to the network. We report these negative results together with the code, data and scripts that reproduce them.**

***Index Terms*—digital twin, software-defined networking, Wi-Fi, large language models, intent-based networking, safe automation**

I. Introduction

Campus Wi-Fi is never still: students move between rooms, a lecture fills a hall within minutes, an AP fails, and neighbouring APs on one channel slow each other down. An SDN controller can react by steering clients to a less loaded AP, changing an AP's channel or prioritizing a traffic class. Reacting is risky, because a controller that applies a change directly learns its effect only afterwards, and the risk grows as more decisions are automated, whether by an optimizer or by an LLM that turns a goal such as "video calls in the lab must stay smooth" into policy. LLM output can be wrong, incomplete or unsafe, and nothing checks it before it takes effect.

We make a *digital twin* of the network the gate that every change must pass. The twin mirrors the live state from telemetry and predicts the effect of a candidate action on throughput, latency, loss and fairness before it is applied. We ask whether this makes an AI-driven controller both more effective and safer than acting directly, and whether an LLM can drive it from natural language. The closed loop of the campaign runs a heuristic optimizer; the LLM pipeline is evaluated separately (H3), and V3 combines the twin with an operator gate for high-impact actions. We test three hypotheses: **H1:** in each of three disruption scenarios (congestion, AP failure, interference) the mean time to recover of the full system (V3) is at least 30% below that of plain SDN (V1), a run that never recovers counting at its end (the capped time; plain SDN never recovers, so its true time is longer, which favours V3); **H2:** the twin discriminates: it rejects more than half of the harmful actions that an optimizer acting directly (V2) applies (we also report the weak form, that it rejects at least one); **H3:** an LLM with schema-constrained output translates at least 20 of 30 intents correctly. That no invalid configuration reaches the network is not a hypothesis: the schema, the compiler and the executor enforce it, and we report it as a property of syntax and bounds, not of fidelity to the operator's goal. The thresholds, 30% and 20 of 30, are targets we set; the second is a low floor, so H3 is weak by design. V1 is plain SDN, the baseline without control; 802.11k/v/r roaming is not emulated (Section VIII).

Contributions. (i) A reproducible, fully virtual wireless SDN testbed with seeded campus scenarios and an explicit model of co-channel interference, which the emulator does not produce itself. (ii) An analytical twin of airtime sharing and queueing, validated against the testbed for steady-state per-flow throughput, compared with the live KPIs in the closed loop, and fast enough for a 5 s control loop. (iii) A closed loop

(propose, verify, apply, watch, roll back) in which the twin's accepted verdict is enforced in the code that applies changes. (iv) An LLM intent pipeline in which the model only translates text into a validated policy; deterministic code compiles it into bounded actions. (v) An evaluation of three controllers over 27 runs that reports what did not work.

II. Related Work

Network digital twins. Almasan et al. [1] describe the network digital twin as an accurate, data-driven model of a real network, used chiefly to answer "what if" questions before a change. We share the split between a mirror of the live state and a performance model, but our model is analytical: it needs no neural training and can be explained, yet a few parameters (capacity, delay constants, queue size) are calibrated on the target testbed. We use the twin as a *safety gate*: no action is applied without its accepted verdict. The loop itself follows the monitor–analyse–plan–execute pattern of autonomic computing [2]. *Learned performance models.* RouteNet [3] uses a graph neural network to predict per-path delay, jitter and loss and to score routing candidates. We follow its use of a fast model inside the loop and its relative-error reporting, but our bottleneck is the shared Wi-Fi medium, which we model explicitly with a fluid airtime model rather than a learned one (RouteNet-Fermi [4] extends the approach to queueing networks; a learned surrogate is future work). Classical 802.11 analyses model the contention itself: Bianchi's Markov chain [5] for saturated CSMA/CA and the performance anomaly of mixed bit rates [6]. Our model needs only per-AP capacity and distance, but it ignores CSMA/CA backoff and the anomaly, a simplification we state in Section VIII. We kept the model small on purpose, to test whether a minimal steady-state model suffices for 5 s control. *LLM-based intent networking.* Mekrache et al. [7] manage the intent life cycle in natural language on a 5G testbed. We adopt natural language as the operator interface, but trust the model less: its output must pass a JSON schema, is compiled by deterministic code and is simulated in the twin, and the model has no tool that changes the network. Intent-based networking is defined in IETF RFC 9315 [8], and NetConfEval [9] benchmarks LLM configuration synthesis for wired routing and device configuration; our 30 intents ask instead for wireless QoS and client-steering policies, so that benchmark could not be reused as it is. Verifying configurations before deployment is established in wired networks (Batfish [10], VeriFlow [11]); those verifiers check logical invariants, whereas ours predicts wireless performance KPIs. *Programmable WLANs.* Odin [12] makes enterprise WLANs programmable through a per-client virtual AP, because clients, not the infrastructure, choose their AP. We steer clients by explicit re-association, a transient that the twin does not model (the watch ignores its first 10 s). IEEE 802.11k/v/r [13] can steer clients natively (BSS transition management); in our emulation stations re-associate when their AP goes down but never roam away from an overloaded or interfered one on their own, so V1 leaves them there, and a V1 with native steering is a further baseline we do not evaluate (Section VIII). *Emulation.* We build on Mininet [14], Mininet-WiFi [15] and OpenFlow [16] with the Ryu controller. None of these works combines a wireless twin, natural-language control and a verification step before every action.

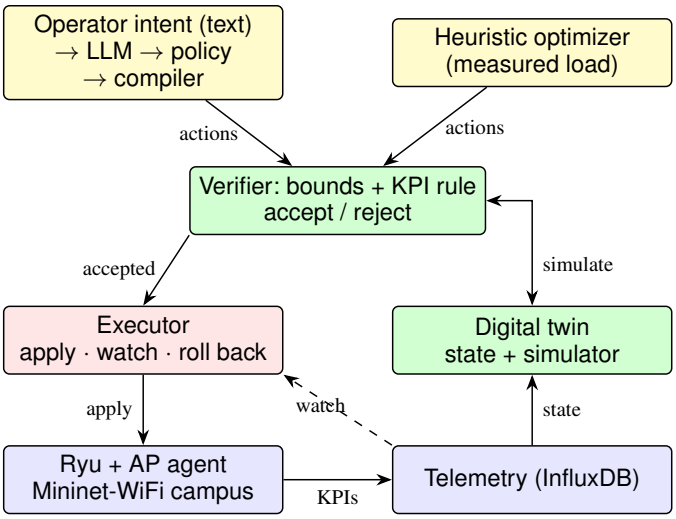


Fig. 1. Architecture. Proposals come from a heuristic optimizer or from an LLM-compiled policy. Only actions that the verifier accepts reach the executor, the only component that changes the network.

III. Problem Scenario and System Model

A. Scenario

The campus has four zones, each served by one AP: a lecture hall (ap1, channel 1), a lab (ap2, channel 6), a corridor (ap3, channel 11) and a library (ap4, channel 1), two OpenFlow switches and 20 stations carrying video, web and bulk traffic. ap1 and ap4 share channel 1 but are 50 m apart (the APs are at ap1 (20, 50), ap2 (60, 50), ap3 (20, 20) and ap4 (60, 20) m), so by Eq. (1) each starts every run with 3.45 Mbit/s instead of 4.6 Mbit/s. An AP carries about 4.6 Mbit/s downlink on our testbed. Each of the three 600 s scenarios has one disruption: a *lecture flash crowd* (ten students walk into the hall from 120 s, video starts at 210 s), an *AP failure* (ap2 goes down at 240 s) and *co-channel interference* (ap3 is forced onto channel 1 at 180 s). The problem is to restore service quickly using only allow-listed actions, without making things worse for users.

B. System model

Fig. 1 shows the architecture. The testbed streams AP, station and flow telemetry to a time-series store. The twin rebuilds its state s from the last 10 s of telemetry at each refresh (no accumulated state, so it cannot drift from the network); an AP silent for 5 s counts as down. The state holds every AP (position, channel, up/down, utilization), station (AP, zone) and flow (measured throughput, latency, loss, queue, rate limit).

An *action* a is one of seven allow-listed types (reroute, QoS queue, rate limit, client steering, transmit power, channel change, AP admin state), each with an impact class. QoS and

rerouting (low impact) and client steering (medium impact, validated against the testbed) are applied when the twin accepts them; the other medium-impact types (rate limit, transmit power) wait for an operator until their predictions are validated; a channel change or taking an AP down is *high impact* and always waits for an operator. The measured KPI vector of a set of flows is $\mathbf{k} = (T, L_{95}, \bar{\ell}, J)$: mean throughput, 95th-percentile video latency, mean loss and Jain's fairness index [17] of the APs' client counts.

IV. Mathematical Formulation

A. Analytical twin

Capacity. An AP i that is up offers $C = 4.6$ Mbit/s, reduced by every up AP $j \neq i$ on its channel:

$$C_i = \frac{C}{1 + \sum_{j \neq i,\ \mathrm{ch}(j)=\mathrm{ch}(i)} w(d_{ij})}, \tag{1}$$

where d_{ij} is the distance between the APs, $w(d_{ij}) = \min\{1, \max\{0, (d_2 - d_{ij})/(d_2 - d_1)\}\}$, $d_1 = 30$ m (a same-channel AP this close takes a full share) and $d_2 = 60$ m (from here it no longer interferes). The emulator does not produce co-channel contention, so the testbed enforces the same C_i as a downlink rate cap on each AP's traffic-control queue while a co-channel AP is up (the cap is removed otherwise; 4.6 Mbit/s is the emulator's own limit). Two consequences: agreement between twin and testbed on capacity sharing validates our implementation and the sharing and queueing logic, not the physical realism of Eq. (1); and Eq. (1) does not depend on the interferer's load, so an idle co-channel AP counts like a busy one. It also means that, in the twin and in the testbed alike, steering clients away from a co-channel AP or rate-limiting its flows gives its neighbours no capacity back: only a channel change or taking an AP down changes C_i, which is why the interference scenario needs a high-impact action.

Sharing. Each AP serves three queues q in strict priority (priority, best effort, background). Within a queue, flow f with demand λ_f (its source rate, cut to its rate limit) receives the max-min fair share of the capacity left to the queue,

$$x_f = \min(\lambda_f, \theta_q), \qquad \sum_{f \in q} \min(\lambda_f, \theta_q) = \min\Big(R_q, \sum_{f \in q} \lambda_f\Big), \tag{2}$$

where, writing $q' \prec q$ for the queues served before q, $R_q = \max\big(0,\, C_i - \sum_{q' \prec q} \sum_{f \in q'} x_f\big)$ (if the queue's demand fits, $x_f = \lambda_f$ and θ_q is any value of at least the largest demand). The demand λ_f is the class's configured rate (video 1.0, web 0.08, bulk 0.394 Mbit/s) and, for a video flow that has been measured, its received rate divided by $1 - \ell_f$ (what its source must have sent; a flow with no throughput or with 100% loss falls back to its class's configured rate, and loss is capped at 100%; the 0.2% base loss is not subtracted, a slight overestimate of λ_f); a flow is in the best-effort queue unless a QoS action moved it. An inelastic flow (video) loses $\ell_f = 1 - x_f/\lambda_f$ plus a base loss of 0.2%; an elastic one (bulk TCP, offered 0.394 Mbit/s, the median measured on the training runs) slows down and sees only the base loss. A web flow takes part in the sharing and in ρ_q with its configured fetch demand, but the rate it reports is that of one page fetch, 0.8 of the capacity that the other traffic in its own or a higher queue leaves free. The signal a station would get at another AP, used by (S1), follows a log-distance path-loss model (-16 dBm at 1 m, exponent 4.0). The twin records a transmit-power step (at most 3 dB) in its state, but Eqs. (1)–(4) do not depend on it, so it predicts no change and accepts it (it is medium impact, so an operator still has to approve it); a reroute is always rejected, because the campus has one path between any two nodes. The heuristics propose neither, and they are not evaluated here.

Latency. Each queue is modelled by the mean number in an M/M/1/K queue [18] at load $\rho_q = \sum_{q' \preceq q} \sum_{f \in q'} \lambda_f / C_i$, where $q' \preceq q$ are the queues served before q or q itself (its own demand plus that of the queues served before it):

$$d_q = d_0 + \sigma\, L(\rho_q, K), \quad L(\rho, K) = \frac{\rho}{1-\rho} - \frac{(K+1)\rho^{K+1}}{1-\rho^{K+1}}, \tag{3}$$

with $L(1, K) = K/2$, $d_0 = 0.8$ ms, $\sigma = 2.5$ ms and $K = 6$. Eq. (3) is a calibrated surrogate, not an exact priority-queue derivation: σ was fitted to the delay plateau measured at saturation and K bounds the queue, so the predicted delay stays below $d_0 + \sigma K = 15.8$ ms; as baseline and prediction are both capped there, the twin cannot predict a violation of the 50 ms recovery threshold, nor a latency change once both saturate. The recovery criterion (5) is applied to *measured* latency, not to this prediction. The twin's L_{95} is the nearest-rank 95th percentile of the predicted latencies of the video flows the verdict compares (of all flows if none is video); as all flows of a station report the latency of its highest-priority queue, this is a percentile over stations, not over packet delays, and it understates the delay of a station's lower-priority flows. A flow without a working AP is dead (no throughput, 100% loss). Network fairness is

$$J = \frac{(\sum_i n_i)^2}{N \sum_i n_i^2}, \tag{4}$$

over the client counts n_i of the N APs that are up ($J = 1$ when no client is associated). J is network-wide and changes only when clients move (steering) or an AP goes up or down. The simulator maps a state and an action set to predicted KPIs, $\hat{\mathbf{k}} = f(s, a)$.

B. Safe control problem

Let t_d be the time the disruption starts to load the network (the AP failure and the interference event as scheduled; in the flash crowd the onset of video at 210 s, when ap1 becomes overloaded; $t_d = 210$, 240 and 180 s in the order of Section III, and $T_{\mathrm{end}} = 600$ s) and $\mathcal{U}(t)$ the utilization of the affected AP, the traffic it serves divided by C_i (measured as the bits its clients sent and received in the last interval, so at most about 1, uplink included); the affected AP is the one with the

highest mean utilization in the 30 s after t_d (for the AP failure, necessarily a surviving one). The *time to recover* is

$$T_{\mathrm{rec}} = \min\big(\{t \ge t_d :\ \mathcal{U}(\tau) < 0.8 \text{ and } L_{95}(\tau) \le 50\,\mathrm{ms} \\ \text{for all } \tau \in [t, t+15\,\mathrm{s}]\} \cup \{T_{\mathrm{end}}\}\big) - t_d, \quad (5)$$

so a run that never recovers is assigned the time from t_d to the end of the run, $T_{\mathrm{end}} - t_d$ (390, 360 or 420 s). The controller chooses at each 5 s tick a set of actions a_t to minimize T_{rec} subject to: (S1) $a_t \in \mathcal{A}(s_t)$, the hard bounds (a steer moves at most 30% of an AP's clients, only stations on that AP, only to an AP where the predicted signal is at least -75 dBm; a transmit-power step is at most 3 dB; the last AP of a zone is never taken down); (S2) the twin's KPI verdict accepts a_t (the verifier also checks any operator policy's constraints and objectives, omitted here), with δ_k the signed improvement of KPI k in percent (positive is better; for latency and loss lower is better) between $\mathbf{k}(s_t) = f(s_t, \emptyset)$, the twin's own simulated no-action baseline (not the live measurement), and $\hat{\mathbf{k}} = f(s_t, a_t)$, over the flows whose simulated KPIs the action changes (for a steer, those on the source and the target AP, whose loads both change; for a channel change, those on the AP and on every AP sharing its old or new channel; J is network-wide):

$$\text{accept} \iff \Big(\forall k :\ \delta_k \ge -5\Big) \ \vee \ \Big(\exists k :\ \delta_k > 5\Big). \quad (6)$$

Here $\delta_k = 100\,(\hat{k} - k)/|k|$ for throughput and J and $100\,(k - \hat{k})/|k|$ for latency and loss (a relative change, also for loss), $\delta_k = 0$ if $k = \hat{k}$, and $\delta_k = +\infty$ or $-\infty$ if $k = 0 \neq \hat{k}$ is an improvement or a degradation (a loss appearing where there was none is $-\infty$). Eq. (6) is the rule as implemented; it agrees with the recorded verdict of all 114 of 114 resolved actions. It is deliberately permissive, and Section VII-B tries stricter rules on the recorded predictions. As J depends on client counts alone, a steer that evens out the counts can satisfy the second clause by itself; that was how 1 of the 41 harmful actions applied after an accepted verdict was accepted. (S3) A high-impact action, and a medium-impact one other than client steering, is applied only after an operator approves it (an approved action is applied as verified, not re-verified against a later state); (S4) For 30 s after an action the live KPIs of the flows the verdict compared are measured, ignoring the first 10 s; if any KPI is worse than before by more than 10% *and* by more than its noise floor (the 95th percentile of natural worsening in windows without an action, measured on the training runs: 0.18 Mbit/s, 10.9 ms, 0.72 points and 0.02 for T, L_{95}, $\bar{\ell}$ and J), the action is rolled back, without checking (S1) again, so the rollback assumes the topology did not change during the watch.

V. Proposed Methodology

Algorithm 1 is one tick of the loop. The heuristics propose two kinds of changes: *client steering* (an AP at or above utilization 0.8 sheds up to 30% of its clients, rounded down, as one proposal per target AP; the targets are the APs below 0.6, least utilized first, and the stations moved are those with the strongest predicted signal at the target; the budget is shared by the targets, so no station is proposed twice and an AP with fewer than four clients sheds none) and *channel change* (an AP at or above 0.8 that shares its channel moves to the channel with the least distance-weighted co-channel load, the sum of $w(d_{ij})$ over the APs that are up in Eq. (1), among channels 1, 6 and 11, and only if one has a strictly lower load than the current channel, the lowest-numbered among equals). While a change is being watched nothing new is proposed, so effects are not mixed up, and a proposal identical to a just-rolled-back one is not repeated for 120 s.

Algorithm 1 One tick of the twin-gated control loop (every 5 s). Actions compiled from an LLM policy are verified jointly through the API and applied only by an operator's call, not by the loop.

```
finish due watches: if a KPI regressed (S4) roll back, else keep
if an earlier change is still being watched then
    return
end if
s ← twin state from the last 10 s of telemetry
P ← PROPOSE(s)                                   (heuristics)
for each proposal p ∈ P do
    v ← VERIFY(s, p): bounds (S1), simulate k̂ = f(s, p), rule (6); record v
    if v accepted and p needs approval then
        queue p for operator approval (S3)
    else if v accepted and at most 3 applied this tick then
        APPLY(p) through the executor; start its watch
    end if
end for
```

Safety enforcement. The executor is the only component that changes the network. It applies an action only if it is in its append-only ledger with an accepted verdict, has the required approval, has not been applied and is within rate limits (at most 3 actions per tick, one high-impact action per AP per 120 s). The heuristics' proposals are verified and applied one at a time, each against the state of the tick and each with its own watch (at most 3 per tick; they usually propose one, and only 2 ticks of the campaign applied more than one, so interactions between proposals of one tick are not simulated; each action is watched and rolled back on its own measurement, but actions applied together are judged over almost the same window, so a regression cannot be attributed among them). The actions an LLM policy compiles to are verified *jointly* and applied as a whole or not at all, and only when an operator calls the API's apply, never by the loop.

LLM roles. The LLM (a cloud model with an offline local fallback, temperature 0) has three roles and none can change the network. (a) It parses an operator's text into a `Policy` under a JSON schema (scope, objectives, hard constraints), with at most two repair attempts; a deterministic compiler turns the policy into bounded actions, which are verified like any other proposal. (b) A copilot answers questions by calling

TABLE I
COMPARED CONTROLLERS. V2 EXISTS TO MEASURE WHAT THE TWIN PREVENTS; V2 AND V3 ALSO DIFFER IN THE OPERATOR GATE (S3) FOR HIGH-IMPACT ACTIONS, WHICH NO OPERATOR USED.

V1 plain SDN	Controller's L2 learning only; stations re-associate by themselves.
V2 no twin	Heuristic proposals applied directly; the twin's verdict is computed and recorded but ignored; every change is watched and rolled back on regression, as in V3.
V3 full	Proposals verified in the twin; high-impact ones wait for an operator; every change is watched and rolled back on regression.

read-only tools and asking the twin. (c) A root-cause explainer turns an alert and a deterministic evidence summary into a structured diagnosis. Forecasting (Holt's linear smoothing [19] against a moving average) and anomaly detection (an Isolation Forest [20] on 30 s whole-network windows) produce forecasts and alerts for the operator and the explainer; the heuristics' proposals use measured load only, because the forecast did not help (Section VII-C).

VI. BASELINES AND EXPERIMENTAL SETUP

Variants. The same testbed runs under three controllers (Table I). *Scenarios and seeds.* The three scenarios of Section III run with seeds 42, 43 and 44 under each variant: 27 runs of 600 s at one frozen code version. All data are *synthetic*, produced by the emulation; no real-network traces are used. The twin, forecaster and detector are calibrated on a separate dataset of 12 recorded runs without control (the three scenarios and a disruption-free one, seeds 42, 43 and 44 being the training, validation and test split); the twin is validated on its held-out test split, which adds a batch of runs with scripted actions. Station and AP positions, per-class demands and the watch's noise floors are in the package's `config/` files. Other seeds: station positions 7, Isolation Forest 42. The LLM is served by Ollama: the main model is `gpt-oss:120b-cloud` and the offline fallback `qwen2.5:3b`; the intent test set is a fixed list of 30 intents (`genai/eval/intents.jsonl`), scored correct if the policy's zone and classes, objectives and constraints equal the expected ones as sets.

Testbed and parameters. Mininet-WiFi with the Ryu controller (OpenFlow 1.3), 4 APs, 2 switches, 20 stations; per-AP downlink 4.6 Mbit/s. The loop runs every 5 s; the verifier tolerance is 5%, the executor's watch 30 s with a regression threshold of 10%. *Reproducibility.* The submission package contains the code, the recorded datasets, the seeds and the configuration; `make results` regenerates every table and figure of Section VII offline from the recorded runs, `make paper` rebuilds this paper's numbers from them, and `make campaign MODE=V1|V2|V3` re-runs one variant's nine runs on the testbed (about two hours each).

Metrics. For each run: time to recover (5) (a run that never recovers is counted at its end, so its mean is a lower bound); mean throughput and mean loss of the *affected flows* (those of the event AP's stations in the 30 s before the disruption, in the flash crowd the busiest AP's; the cohort is fixed by station identity, so a flow stays in it when it is steered away; in the AP failure it is the failed ap2's stations, whereas $\mathcal{U}$ in Eq. (5) is that of a surviving AP); p95 video latency of those flows (this evaluation cohort is not the set of flows a verdict compares, which is the action's own scope); and Jain's index (4), averaged per 5 s over the stress window, from t_d to the end of the run. We report the mean over the three seeds with their smallest and largest value; a Student-t interval would assume a normal distribution that three seeds, right-censored recovery times and bimodal failures do not give. Fig. 3 and the text give the per-seed values where the range is widest. H1 is called *met* or *not met* on the capped times, and we say where the cap could change that. An action is *resolved* when it was kept, rolled back, or its rollback failed, and *harmful* if the watch measured a regression above 10% (rolled back, or the rollback failed).

VII. RESULTS AND PERFORMANCE ANALYSIS

A. Recovery and network KPIs (H1)

Table II and Fig. 2 give the results. In the *flash crowd*, plain SDN leaves the arriving students on ap1: the affected flows lose 43.9% of their packets and carry 0.20 Mbit/s. Both acting variants steer clients away, raising throughput to 0.52–0.53 Mbit/s and cutting loss to 9.2% (V2) and 8.5% (V3), with ranges that do not overlap V1's. Jain's index rises from 0.51 to 0.78 (V2) and 0.79 (V3), just under the target of 0.8 set for the flash crowd (separate from the recovery criterion). V2 and V3 are not distinguishable here: steering is applied without an operator in both. In the *AP failure*, the stations of ap2 re-associate by themselves, so V1 already works (4.8% loss) and acting adds little; V1 still counts as never recovered: its p95 latency stays near 18 ms, far below 50 ms, so as far as the data show it is the other condition of Eq. (5), the affected AP (a surviving one) under 0.8 utilization, that the stations moved from ap2 prevent. The wide ranges and the p95 latency of V2 and V3 (348 and 672 ms against 18 ms) come from failed rollbacks: 3 of the 3 rollbacks that failed in the campaign were in AP-failure runs, where a steer issued shortly before the failure was judged by its 30 s watch against the failure itself, was rolled back onto the failed AP, and left a station without an AP for the rest of the run. Per seed (42, 43, 44), V2's loss was 25.2, 2.7, 2.3% and its p95 latency 1000, 24, 21 ms, V3's 4.2, 26.1, 26.0% and 15, 1000, 1000 ms: one V2 run and two V3 runs hit the failure (1000 ms is what the probe reports for a ping that gets no reply), so a mean misdescribes a bimodal outcome. In the *interference* scenario the fix is a high-impact channel change. V3's heuristics proposed it and the twin accepted it, but with no operator it was never applied; V3 could only steer clients, which still cut loss from 48.2% to 25.7%. V2 applied the change directly and recovered in every run (mean 146 s). V2 against V3 here therefore compares a loop with an absent operator against one without, not the effect of the twin alone; an auto-approving V3 would isolate it (future work).

TABLE II

KPIs OF THE AFFECTED FLOWS (MEAN AND [SMALLEST–LARGEST] OVER THE THREE SEEDS; WITH SO FEW SEEDS NO INTERVAL IS GIVEN). $\geq$ MARKS A RECOVERY TIME THAT INCLUDES RUNS THAT NEVER RECOVERED (THEIR NUMBER IN BRACKETS), COUNTED AT THE RUN END. A P95 LATENCY OF 1000 MS IS THE PROBE'S VALUE FOR A PING THAT GETS NO REPLY.

Scenario	Variant	Recovery (s)	Thr. (Mbit/s)	p95 lat. (ms)	Loss (%)	Jain
Lecture flash crowd	V1 plain SDN	$\geq$ 390 [390–390] (3)	0.20 [0.20–0.20]	21 [21–22]	43.9 [43.8–44.0]	0.51 [0.51–0.51]
	V2 no twin	$\geq$ 374 [343–390] (2)	0.52 [0.51–0.53]	24 [21–28]	9.2 [7.2–11.1]	0.78 [0.78–0.78]
	V3 full	$\geq$ 359 [313–390] (1)	0.53 [0.50–0.57]	25 [21–28]	8.5 [7.9–9.6]	0.79 [0.77–0.80]
AP failure	V1 plain SDN	$\geq$ 360 [360–360] (3)	0.64 [0.62–0.68]	18 [17–18]	4.8 [4.5–5.3]	0.96 [0.96–0.97]
	V2 no twin	$\geq$ 142 [17–360] (1)	0.84 [0.64–0.96]	348 [21–1000]	10.0 [2.3–25.2]	0.91 [0.88–0.93]
	V3 full	$\geq$ 300 [180–360] (2)	0.58 [0.52–0.67]	672 [15–1000]	18.8 [4.2–26.1]	0.89 [0.88–0.91]
Co-channel interference	V1 plain SDN	$\geq$ 420 [420–420] (3)	0.18 [0.18–0.18]	11 [10–11]	48.2 [48.0–48.3]	0.93 [0.93–0.93]
	V2 no twin	146 [33–352] (0)	0.60 [0.39–0.72]	22 [19–24]	9.1 [1.0–25.0]	0.95 [0.93–0.97]
	V3 full	$\geq$ 420 [420–420] (3)	0.31 [0.30–0.32]	11 [10–12]	25.7 [22.2–27.5]	0.94 [0.94–0.94]

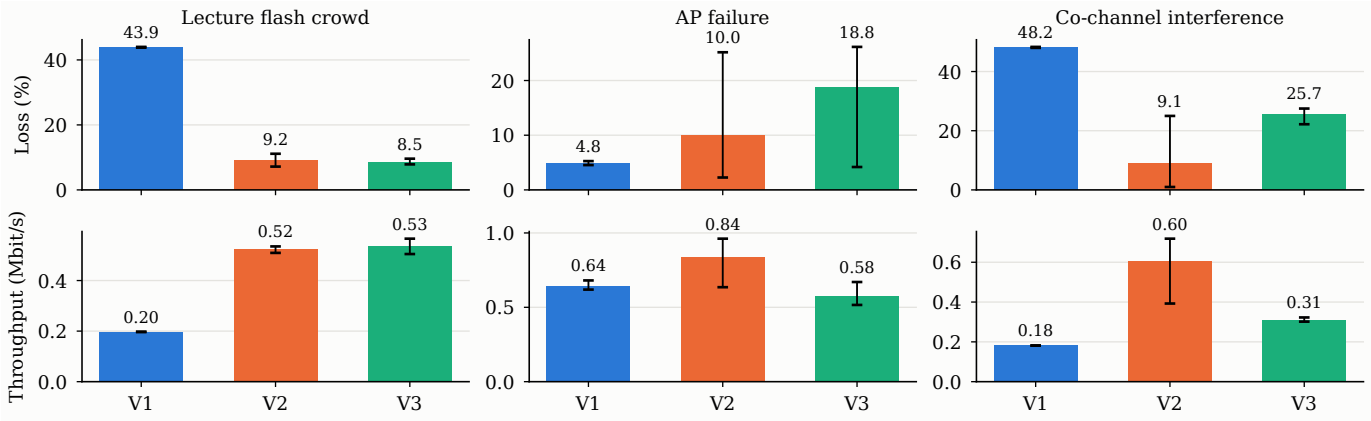


Fig. 2. Loss (top) and throughput (bottom) of the affected flows per scenario and variant; bars are means, whiskers the smallest and largest of the three seeds.

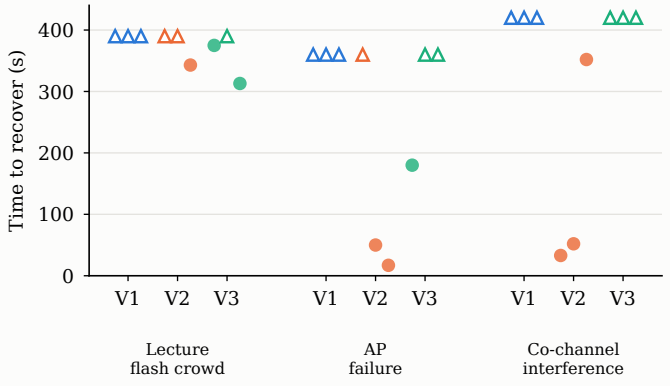


Fig. 3. Time to recover of every run. A hollow triangle is a run that never recovered, plotted at its run end.

H1 is not met on the capped recovery times, in every scenario. V3 is faster than V1 by 8%, 17% and 0% (flash crowd, AP failure, interference; target 30%). Plain SDN never recovered in any of its nine runs (Fig. 3), so the cap understates its true time and a longer horizon could widen the gap where V3 recovered at least once (flash crowd, AP failure); in the interference scenario V3 never recovered either, so nothing supports H1 there. Counting runs instead: 0 (V1), 6 (V2) and 3 (V3) of nine recovered.

B. Safety and the twin's role (H2)

Table III gives what became of every action, from the executor ledgers of the passed runs. In V2 the twin rejected 3 actions that were applied anyway (2 were then rolled back), and in V3 it rejected 4, none of which reached the network. Only V2 allows that comparison, since it applies every action whatever the verdict (V3 applies accepted actions alone). Of V2's 23 harmful actions (rolled back, or the rollback failed) the twin had rejected 2 and accepted 21; V3 applied only accepted actions, and 20 of them were harmful, so 41 harmful actions were applied after an accepted verdict in all (the 43 harmful actions of V2 and V3 less the 2 the twin rejected). The harm rate among V2's resolved actions was 2 of 3 for twin-rejected and 21 of 58 for twin-accepted actions: higher, but not significantly so (one-sided Fisher exact test, $p = 0.32$; the cells are tiny and the actions of one run are not independent, so even this p-value is optimistic). **H2 is not met**: the twin rejected 2 of V2's 23 harmful actions, far from a majority, and the harm rates do not differ significantly (the weak form, that it rejects at least one, holds trivially and says little). A scheduled fault could inflate the harm counts, because the watch cannot tell it

TABLE III

FATE OF EVERY ACTION IN THE PASSED RUNS (NINE RUNS PER VARIANT), FROM THE EXECUTOR LEDGERS. V2 APPLIES ACTIONS WHATEVER THE TWIN SAID (THE TWIN HAD REJECTED 3 OF THEM), SO NONE SHOWS AS REJECTED THERE. V2'S 61 RESOLVED ACTIONS ARE 23 HARMFUL (22 ROLLED BACK, 1 FAILED ROLLBACK) AND 38 KEPT; THE TWIN HAD REJECTED 2 HARMFUL AND 1 KEPT. THE 41 HARMFUL ACTIONS APPLIED AFTER AN ACCEPTED VERDICT ARE V2'S 21 AND V3'S 20.

Actions	V1	V2	V3
Proposed	0	64	65
Kept	0	38	33
Rolled back	0	22	18
Rollback failed	0	1	2
Still watched	0	3	0
Rejected, not applied	0	0 (3 ignored)	4
Awaiting an operator	0	0	8

from the action's effect: for 18 of the 43 harmful actions of V2 and V3 the scenario's disruption fell within the 30 s before or after the action, the window the watch compares (among them the 3 failed rollbacks). Leaving those out, the twin still rejected only 2 of the 15 remaining harmful actions of V2, so the fault does not explain the misses. 7 of V2's 38 kept actions were judged across it too, and without all the tainted actions (87 remain) the closed-loop errors are 55.6%, 76.8% and 59.0% against 50.5%, 66.0% and 48.5% for "nothing changes".

Would a stricter rule help? Eq. (6) accepts an action if any KPI is predicted more than 5% better. Table IV applies stricter rules to the recorded predictions of the same actions. Even rejecting every predicted regression above 5% would stop only 4 of V2's 23 harmful actions, and also 4 of its 38 kept ones: only 11 of the 23 harmful actions had any predicted regression at all (4 of more than 5%), so the weakness is the prediction, not the rule. Gating on only the two KPIs whose prediction was validated, throughput and Jain's index, would have rejected 2 of the 23 and 3 of the 38: no better. Against the live KPIs measured in the watch, the twin's median relative error (over the resolved actions with a nonzero measurement: 114 for throughput, 114 for latency, 102 for loss and 114 for Jain's index, V2 and V3 together) is 56.7% for throughput, 70.5% for latency and 57.2% for loss, not better than the twin's own simulated "nothing changes" baseline (46.5%, 62.3% and 50.3%); only Jain's index, a function of client counts, is predicted exactly (0.0% against 2.7%). The watch window includes the transient the twin does not model. The steady-state accuracy of Section VII-C therefore does not carry over to the closed loop, and the latency and loss predictions were never validated separately.

The main protection was the watch-and-rollback step, and it is not guaranteed: 3 of the 43 rollbacks that were attempted failed. In V3 every applied action had an accepted verdict and no high-impact change was applied without an operator (V2: 9 such changes), which is why V3 could not repair the interference unattended; 8 accepted changes were still waiting for approval when their runs ended.

TABLE IV

STRICTER ACCEPTANCE RULES APPLIED TO THE RECORDED PREDICTIONS OF V2'S RESOLVED ACTIONS: HOW MANY HARMFUL ACTIONS EACH WOULD HAVE REJECTED, AND HOW MANY KEPT ONES.

Acceptance rule	Harmful rejected	Kept rejected
As written (no KPI >5% worse, or one >5% better)	2 of 23	1 of 38
No KPI more than 5% worse	4 of 23	4 of 38
No KPI more than 5% worse and one more than 5% better	4 of 23	6 of 38
As written, on throughput and Jain's index only	2 of 23	3 of 38

C. Twin accuracy, LLM and detection

Twin. On 586 held-out situations (495 of them steady-state video) the twin's mean absolute percentage error for per-flow throughput is 4.5%: 4.0% in steady state, 3.3% after client steering and 8.1% after a channel change (only 10 cases), while bulk TCP flows are weakest (12–21%). A prediction takes a median 0.61 ms. *LLM (H3).* On a 30-intent test set the cloud model translated 30 of 30 correctly (median 2.0 s) and the local fallback 21 of 30 (median 2.5 s), so **H3 holds**; of the 30 intents 17 ask for one objective in one class and 13 are compound (a second objective, a hard constraint or two classes); the cloud model translated all of both kinds, the local one all 17 simple intents but only 4 of the compound ones, and its 9 misses all passed the schema: a semantic error that neither schema nor compiler can catch, so an operator must read the compiled actions; 60 of the 60 calls of the two evaluations were schema-valid at the first attempt (0 repairs were needed), every output passes the schema before the compiler sees it, and no invalid output reached the network. On live faults the copilot answered 5 of 5 diagnostic questions correctly, the explainer diagnosed 3 of 3 faults from the first alert, and all 6 of 6 LLM-proposed actions have their twin verdict in the audit log (the explainer only suggests them, and none was applied). *Detection.* The Isolation Forest reaches precision 0.950, recall 0.979 and F1 0.964 on the test split and flags the AP failure 5 s after it occurs. Holt's trend method does not beat the moving average (RMSE 0.129 against 0.117 at one minute, 0.256 against 0.212 at three), because load changes in steps; the loop therefore reacts to measured load rather than to the forecast.

VIII. LIMITATIONS

The network is emulated on one machine with a fixed propagation model, and the numbers describe this emulation, not a real campus. The co-channel interference the twin models is the model the testbed enforces (Section IV-A), so the twin is not validated against independent contention physics, and Eq. (1) ignores CSMA/CA backoff, the 802.11 performance anomaly and the interferer's load. The twin is calibrated on this testbed, predicts steady state rather than the transient, and its closed-loop predictions are no better than "nothing changes" (Section VII-B); its latency and loss predictions were

not validated separately and Eq. (3) cannot predict more than 15.8 ms. The acceptance rule (6) is permissive, though stricter rules would not have helped; proposals of one tick are verified one by one. Three seeds give only ranges, not intervals, and the evaluation seeds reuse the seed numbers of the calibration dataset (the runs are separate, and in V2 and V3 the controller acts while V1 does not); a fresh set of seeds is the obvious next campaign, as is a V1 with native 802.11k/v/r steering: without client-side roaming V1 cannot recover in the flash crowd or the interference scenario within the 600 s horizon, which is why H1 is decided on capped times. The campaign was unattended, so V3 never applied a high-impact action; V2 and V3 therefore differ in the twin and in the operator gate. An auto-approving V3 without delay, or a V2 that keeps the operator gate, would isolate the twin's effect (a human-scale delay would confound it again). H3 was tested outside the loop, on a small internal intent set on which the cloud model scored 30 of 30, not on a standard benchmark such as NetConfEval nor on ambiguous or adversarial intents, and we did not run the local model's semantic misses through the verifier. Four fixes follow from the failed rollbacks and the stale approvals: a rollback should never target an AP that is down; a watch should not judge an action while an AP goes silent or down; cancelling a watch on such an event should release the loop, which proposes nothing while a change is watched, so that it can react on the next tick; and an approved action should be verified again against the state at the time it is applied. Suspending the watch on any anomaly would be unsafe, because an outside surge and an action's own congestion look alike; only verifiable infrastructure events should cancel it.

IX. Conclusion

Used as a gate, the twin kept every applied action behind an accepted verdict and every high-impact change behind an operator, and an LLM behind that gate stated goals in natural language without ever acting on its own: no invalid output reached the network. That the twin makes control *safer in outcome* is only partly shown: in V2 it rejected 2 of 23 harmful actions (and 20 of V3's applied actions, all accepted, were harmful), its closed-loop predictions were no better than "nothing changes", a stricter acceptance rule would not have helped, the watch-and-rollback step did most of the protecting, and in the AP-failure scenario V3's flows fared worse than plain SDN's. Whether the loop recovers the network faster depends on the fault and on having an operator for the changes that matter most; H1 was not met, and the evidence is mixed. We measured each failure instead of assuming it: the steers that hurt, the rollbacks that failed and the changes left waiting. Next steps are a rollback that avoids failed APs, a twin whose latency, loss and transient predictions are validated (learned or analytical), a campaign on fresh seeds with an auto-approving operator, and an evaluation on real access points.

Acknowledgment

Parts of the code and of the text were drafted with the help of an AI coding assistant (Claude Code). The authors reviewed all content and take responsibility for it.

References

[1] P. Almasan, M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, D. Perino, D. López, A. A. P. Perales, P. Harvey, L. Ciavaglia, L. Wong, V. Ram, S. Xiao, X. Shi, X. Cheng, A. Cabellos-Aparicio, and P. Barlet-Ros, "Network digital twin: Context, enabling technologies, and opportunities," *IEEE Commun. Mag.*, vol. 60, no. 11, pp. 22–27, Nov. 2022.

[2] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003.

[3] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2260–2270, Oct. 2020.

[4] M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, K. Rusek, S. Xiao, X. Shi, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet-Fermi: Network modeling with graph neural networks," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3080–3095, Dec. 2023.

[5] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE J. Sel. Areas Commun.*, vol. 18, no. 3, pp. 535–547, Mar. 2000.

[6] M. Heusse, F. Rousseau, G. Berger-Sabbatel, and A. Duda, "Performance anomaly of 802.11b," in *Proc. IEEE INFOCOM*, vol. 2, San Francisco, CA, USA, 2003, pp. 836–843.

[7] A. Mekrache, A. Ksentini, and C. Verikoukis, "Intent-based management of next-generation networks: an LLM-centric approach," *IEEE Netw.*, vol. 38, no. 5, pp. 29–36, Sep. 2024.

[8] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-based networking – concepts and definitions," IETF, RFC 9315, Oct. 2022.

[9] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs facilitate network configuration?" *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, pp. 7:1–7:25, Jun. 2024.

[10] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proc. 12th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, Oakland, CA, USA, 2015, pp. 469–483.

[11] A. Khurshid, X. Zou, W. Zhou, M. Caesar, and P. B. Godfrey, "VeriFlow: Verifying network-wide invariants in real time," in *Proc. 10th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, Lombard, IL, USA, 2013, pp. 15–27.

[12] L. Suresh, J. Schulz-Zander, R. Merz, A. Feldmann, and T. Vazão, "Towards programmable enterprise WLANs with Odin," in *Proc. 1st Workshop on Hot Topics in Software Defined Networks (HotSDN)*, Helsinki, Finland, 2012, pp. 115–120.

[13] IEEE, "IEEE standard for information technology – telecommunications and information exchange between systems – local and metropolitan area networks – specific requirements – part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications," IEEE Std 802.11-2020, Feb. 2021.

[14] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, Monterey, CA, USA, Oct. 2010, pp. 19:1–19:6.

[15] R. R. Fontes, S. Afzal, S. H. B. Brito, M. A. S. Santos, and C. E. Rothenberg, "Mininet-WiFi: Emulating software-defined wireless networks," in *Proc. 11th Int. Conf. Netw. Service Manag. (CNSM)*, Barcelona, Spain, Nov. 2015, pp. 384–389.

[16] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Apr. 2008.

[17] R. Jain, D.-M. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Hudson, MA, USA, Tech. Rep. DEC-TR-301, Sep. 1984.

[18] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley, 1975.

[19] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *Int. J. Forecasting*, vol. 20, no. 1, pp. 5–10, 2004.

[20] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.